

Arquitectura de referencia

Sistema de scoring de salud financiera de PYMEs
sobre datos de tesorería (HackSpain — reto Embat)

El Mamoun Bellachab

September 18, 2026

Abstract

Este documento define la arquitectura de un sistema de *machine learning* que estima la probabilidad de deterioro financiero de una empresa a partir de su actividad de tesorería (cuentas bancarias, financiación, transacciones y facturas), sin usar RAG ni LLMs en el camino crítico de la predicción. Se describen once capas repartidas en dos flujos — *offline* (entrenamiento) y *online* (servicio) —, con modelos candidatos, ventajas e inconvenientes y recomendación explícita en cada una. El documento parte de la exploración real del dataset entregado y documenta dos hallazgos que condicionan todo el diseño: la **ausencia de variable objetivo** y la **inexistencia de grafo entre empresas**.

Contents

1	Punto de partida: qué dicen realmente los datos	2
1.1	Hallazgo 1: no existe variable objetivo	2
1.2	Hallazgo 2: no hay grafo entre empresas	2
1.3	Hallazgo 3: el volumen obliga a elegir motor	3
1.4	Hallazgo 4: hay datos “sucios” que son señal	3
2	La tesis del proyecto	3
3	Vista global de la arquitectura	3
4	Capa 1 — Ingestión	5
5	Capa 2 — Modelo de datos	5
6	Capa 3 — Etiquetado (target engineering)	6
7	Capa 4 — Ingeniería de <i>features</i>	8
8	Capa 5 — Entrenamiento y validación	9
9	Capa 6 — Evaluación y explicabilidad	11
10	Capa 7 — Registro de modelos	11
11	Capa 8 — Base de datos servida	12
12	Capa 9 — Servicio de inferencia	13
13	Capa 10 — API / Backend	13

14

Capa 11 — Frontend

14

15

Capa 12 — Infraestructura, reproducibilidad y MLOps

15

16

Riesgos y decisiones abiertas

15

17

Orden de construcción sugerido

15

1 Punto de partida: qué dicen realmente los datos

Antes de proponer capas conviene fijar lo que hay. El dataset son nueve ficheros CSV (≈ 617 MB) con 1.286 empresas agrupadas en 250 grupos empresariales y 24 meses de historia (2024-09-01 a 2026-09-01).

Fichero	Grano	Filas	Rol en el modelo
groups.csv	grupo	250	jerarquía / contagio
companies.csv	empresa	1.286	entidad principal
banking_products	cuenta	5.987	estructura bancaria
debt_products	facilidad	2.239	apalancamiento
debt_schedule_config	préstamo	87	condiciones (tipo, plazo)
balances.csv	producto	7.996	foto de liquidez
invoices.csv	factura	897.894	comportamiento de pago
transactions.csv	movimiento	2.556.437	flujo de caja

1.1 Hallazgo 1: no existe variable objetivo

Ningún fichero contiene un campo de quiebra, default, impago estructural ni rating. No es un dataset supervisado. Por tanto **el target hay que construirlo**, y esa construcción es la decisión de diseño con más impacto en el resultado final: un modelo excelente sobre una etiqueta mal definida no predice nada útil. Se le dedica la Capa 3.

1.2 Hallazgo 2: no hay grafo entre empresas

La intuición inicial era explotar la red de contrapartidas (“si mi cliente cae, yo caigo”). Los datos no lo permiten:

Fuente	Contrapartidas	Compartidas por > 1 empresa
invoices.csv	124.030	0
transactions.csv	47.796	91 (máx. 2 empresas)

Los espacios de contrapartidas son prácticamente disjuntos por empresa. Una arquitectura basada en *Graph Neural Networks* sobre un grafo empresa–empresa entrenaría sobre componentes conexas de tamaño 1: no aporta señal. La estructura relacional que sí existe y sí es explotable es otra:

- Grupo** $\rightarrow$ **empresa**: holdings de hasta 24 filiales; el deterioro de una filial es informativo sobre las hermanas (contagio intragrupo real).
- Empresa** $\rightarrow$ **banco**: empresas que comparten entidad financiera comparten shock de crédito y política de riesgo.
- Empresa** $\rightarrow$ **contrapartida (bipartito intra-empresa)**: no conecta empresas entre sí, pero mide *concentración* de clientes/proveedores, que sí es un predictor clásico de fragilidad.

1.3 Hallazgo 3: el volumen obliga a elegir motor

2,5 M de transacciones y 900 k facturas no se procesan cómodamente con **pandas** puro en un portátil durante un hackathon. La capa de ingestión debe convertir a Parquet columnar y agregar con un motor OLAP embebido.

1.4 Hallazgo 4: hay datos “sucios” que son señal

El 21,5% de los productos tienen saldo negativo; el 21,4% de las facturas están en estado **overdue**; el 90% de las transacciones no tienen contrapartida resuelta; el campo **country** está frecuentemente vacío. **La ausencia no es ruido, es información**: una empresa cuya conciliación contable empeora, o cuyo ERP deja de sincronizar, es una empresa que está perdiendo el control de su tesorería. Los indicadores de *missingness* entran como *features*, no se imputan silenciosamente.

2 La tesis del proyecto

La hipótesis que defendemos — y que el sistema debe demostrar empíricamente — es:

El scoring crediticio clásico (tipo FICO empresarial, CIRBE, rating de agencia) se construye sobre estados financieros anuales y sobre incidencias de impago ya declaradas. Es tardío y de baja frecuencia. La tesorería diaria contiene señales de comportamiento que anticipan el deterioro meses antes: retrasos crecientes en el pago a proveedores, nóminas que se pagan tarde, impuestos aplazados, uso creciente del descubierto, concentración de cobros en pocos clientes, y caída de la ratio cobros/pagos. Estas variables no aparecen en un balance y son justamente las que un banco no ve.

De ahí la estrategia experimental: entrenar dos modelos sobre la misma etiqueta y el mismo *split* temporal.

	Modelo A — “proxy FICO”	Modelo B — “comportamental”
Entrada	solo variables tipo balance: deuda, saldo, antigüedad, sector	A + bloques de comportamiento (timing, concentración, grupo, texto)
Rol	<i>baseline</i> a batir	propuesta

La entrega no es “tenemos un modelo con 0,89 de AUC”; es “**el lift de B sobre A es de +X puntos de AUC-PR, y SHAP demuestra que ese lift viene de los bloques de comportamiento**”. Eso es lo que hace la demo defendible ante un jurado que sabe de finanzas.

3 Vista global de la arquitectura

El sistema se organiza en **once capas** repartidas en dos flujos. El flujo *offline* se ejecuta sin usuario delante, procesa los CSV históricos y produce un artefacto: el modelo entrenado más la tabla de scores. El flujo *online* se ejecuta en vivo, en milisegundos, y solo lee. **El registro de modelos y la base de datos son la bisagra**: el flujo offline escribe en ellos, el online lee de ellos.

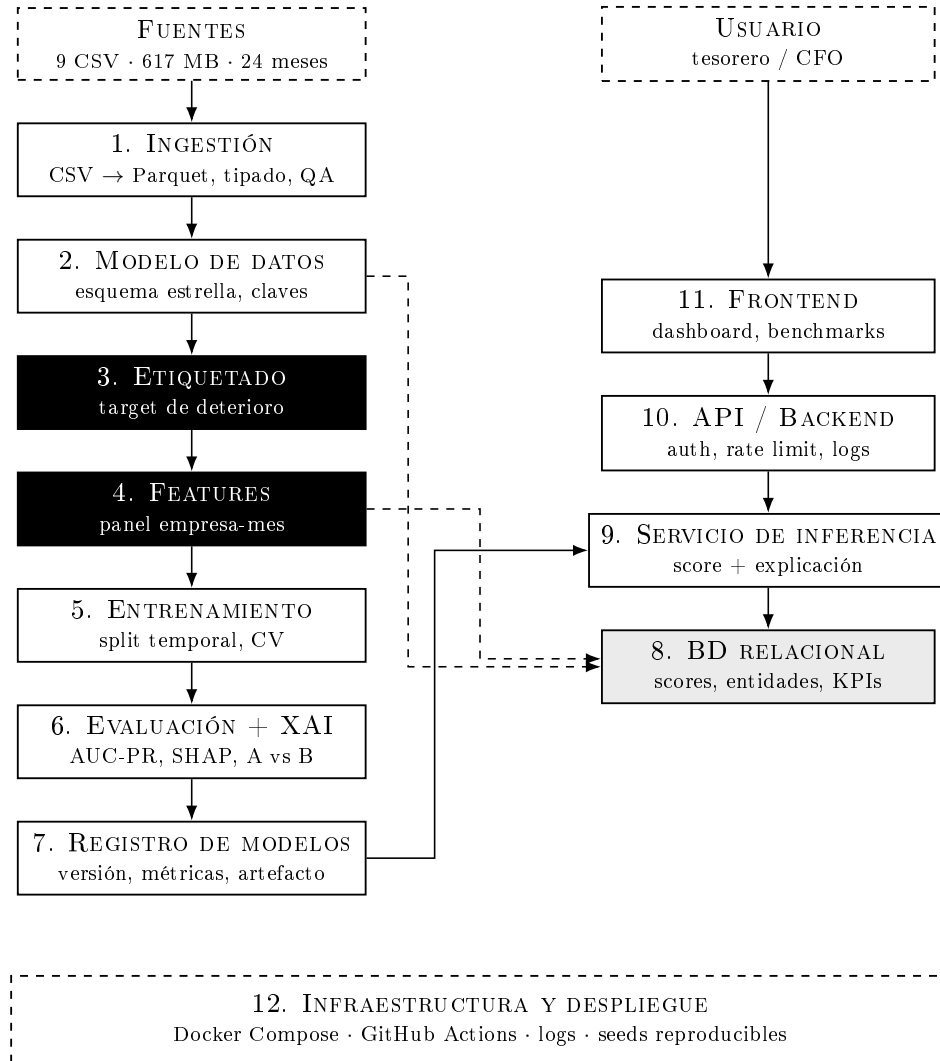


Figure 1: Arquitectura general. Flujo *offline* a la izquierda (batch, sin usuario), flujo *online* a la derecha (síncrono). Las cajas negras son las dos capas donde se concentra el valor diferencial del proyecto. Las flechas discontinuas son escrituras batch hacia la base de datos servida.

4 Capa 1 — Ingestión

Función

Convertir nueve CSV heterogéneos en un almacén columnar tipado, particionado y reproducible. Es la única capa que toca los ficheros originales; a partir de aquí nadie vuelve a leer un CSV.

Detalle técnico

- Lectura en *streaming* por *chunks* (`transactions.csv` son 472 MB; leerlo entero en memoria con tipos por defecto consume varios GB).
- Tipado explícito: fechas a `datetime64[ns]`, importes a `float64`, IDs a `category`. Nunca inferencia automática — rompe la reproducibilidad.
- Normalización de terminadores (`\r\n`) y de codificación.
- Escritura a Parquet con compresión `zstd`, particionado por mes en las tablas de hechos.
- **Capa de calidad:** conteo de filas, nulos por columna, integridad referencial (`company_id` huérfanos), rangos de fecha. Se persiste como informe versionado.

Opciones

Opción	A favor	En contra
pandas directo	Familiar, cero setup	Se ahoga con 2,5 M filas × joins; RAM
DuckDB	SQL sobre CSV/Parquet sin cargar en RAM; agregaciones 10–50× más rápidas; cero servidor	Menos conocido; menos ecosistema de gráficos
polars	Muy rápido, API moderna, <i>lazy</i>	API distinta a pandas: curva de aprendizaje en hackathon
PySpark	Escala real	Sobredimensionado: overhead de JVM para 617 MB

Recomendación

DuckDB + Parquet. Permite escribir las agregaciones pesadas (cobros mensuales por empresa, DPD medio por mes) en SQL plano, que es más rápido de escribir y de revisar que encadenar `groupby` de pandas. Se pasa a pandas solo en el último paso, cuando la tabla ya es pequeña (~30k filas). Evitar Spark: el coste de arranque no se amortiza.

Riesgo principal

Silenciar errores de parseo. Si una fecha mal formada se convierte en `NaT` sin aviso, la ventana temporal del etiquetado se corrompe y no se detecta hasta el final. Todo descarte debe quedar registrado con conteo.

5 Capa 2 — Modelo de datos

Función

Fijar un esquema relacional explícito sobre el que razonan todas las capas posteriores. Sin esto, cada notebook reinventa sus propios *joins* y los resultados dejan de ser comparables.

Decisión de diseño: esquema estrella

- **Dimensiones** (lentas, pequeñas): `dim_company`, `dim_group`, `dim_product`, `dim_counterparty`, `dim_bank`.

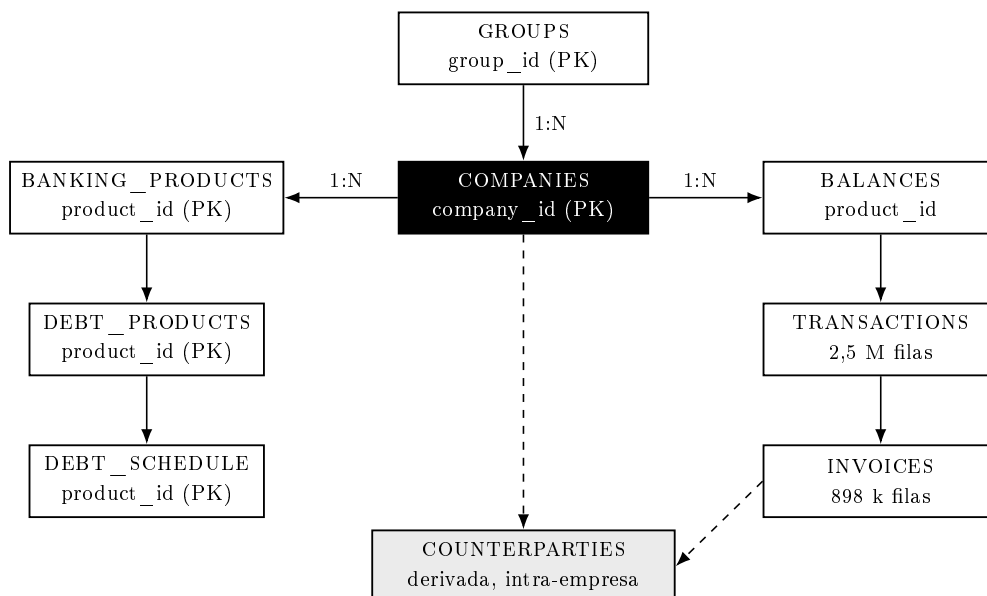


Figure 2: Modelo entidad-relación. `company_id` es la clave que atraviesa todos los ficheros. La tabla de contrapartidas es derivada: se materializa para calcular concentración, pero *no* conecta empresas entre sí (ver Hallazgo 2).

- **Hechos** (grandes, con fecha): `fact_transaction`, `fact_invoice`, `fact_balance`.
- **Tabla derivada central:** `fact_company_month` — el panel empresa×mes que alimenta al modelo. $1.286 \times 24 \approx 30.864$ filas.

Opciones de motor

Opción	A favor	En contra
SQLite	Cero configuración, fichero único	Tipado débil, concurrencia limitada
PostgreSQL	Estándar, tipos ricos, ventanas SQL, se despliega igual en local y cloud	Requiere contenedor
DuckDB como BD servida	Ya está en la capa 1	Pensado para analítica, no para servir concurrencia

Recomendación

DuckDB para transformar, PostgreSQL para servir. Es la separación estándar OLAP/OLTP y evita el error típico de servir un dashboard leyendo Parquet a cada clic. Postgres guarda las tablas finales pequeñas: entidades, scores, KPIs de benchmark.

6 Capa 3 — Etiquetado (target engineering)

Por qué esta capa es la más importante

No hay etiqueta. Si se define mal, todo lo demás es irrelevante. Y hay una trampa clásica: **si la etiqueta se construye con variables que luego se usan como *features*, el modelo alcanza AUC 0,99 y no ha aprendido nada** — ha memorizado la fórmula de la etiqueta. Esto se llama fuga de etiqueta (*label leakage*) y es el fallo que más proyectos de hackathon hunde en la ronda de preguntas.

Marco temporal

Se adopta un esquema de **ventana de observación / gap / ventana de resultado**:

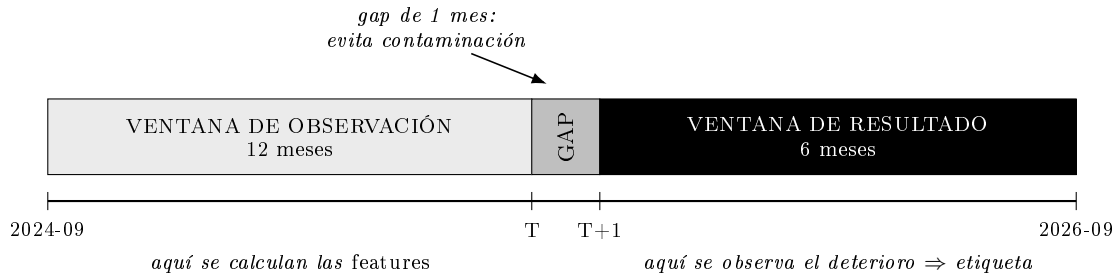


Figure 3: Diseño temporal del etiquetado. Ninguna variable calculada después de T puede entrar como *feature*. El *gap* protege frente a información que se registra con retraso contable.

Definición del target propuesta

Al no haber quiebra observable, se construye un **índice compuesto de deterioro** sobre la ventana de resultado, y se binariza por percentil. Componentes candidatos:

Señal	Cálculo	Interpretación
Morosidad emergente	% facturas overdue y $DPD > 30$	la empresa deja de pagar
Iliquidez sostenida	meses con saldo agregado < 0	descubierto estructural
Colapso de cobros	caída $> 40\%$ en cobros vs. ventana previa	pérdida de ingresos
Estrés de deuda	outstanding/granted $\rightarrow 1$	agotamiento de líneas
Cese de actividad	sin transacciones ≥ 2 meses	abandono / cierre

Formalmente, con z -scores por sector/tamaño para evitar que el tamaño domine:

$$D_i = \sum_k w_k z_{ik}, \quad y_i = \mathbb{I}[D_i > Q_{0.85}(D)]$$

Alternativas de formulación

Formulación	A favor	En contra
Binaria (sano/riesgo)	Simple, métricas estándar, fácil de explicar	Umbral arbitrario; pierde matiz
Multiclase (3 niveles)	Semáforo natural para el dashboard	Fronteras aún más arbitrarias
Regresión sobre D	Sin umbral, conserva ordenación	Más difícil de comunicar
Ranking / score continuo	Es lo que un tesorero usa de verdad (ordenar cartera)	Requiere métricas de ranking
Supervivencia (Cox)	Modela <i>cuándo</i> , no solo <i>si</i>	Sobredimensionado para 24 meses

Recomendación

Binaria para entrenar y evaluar, score continuo (probabilidad calibrada) para mostrar. Se entrena clasificación binaria porque las métricas son inequívocas, pero el dashboard nunca enseña “sí/no”: enseña una probabilidad y un percentil frente al sector. Documentar la fórmula de D en el README como decisión explícita y defendible.

Disciplina anti-leakage (regla dura)

Toda variable usada en D queda **prohibida** como *feature* en su forma directa dentro de la ventana de resultado. Las mismas magnitudes sí se pueden usar *medidas en la ventana de observación*: el DPD de hoy predice el impago de dentro de seis meses; el DPD de dentro de seis meses *es* el impago.

7 Capa 4 — Ingeniería de *features*

Función

Convertir 3,4 M de eventos en una matriz $\approx 30.000 \times 120$. Aquí es donde se gana o se pierde el proyecto: con datos tabulares, la calidad de las variables pesa mucho más que la elección del algoritmo.

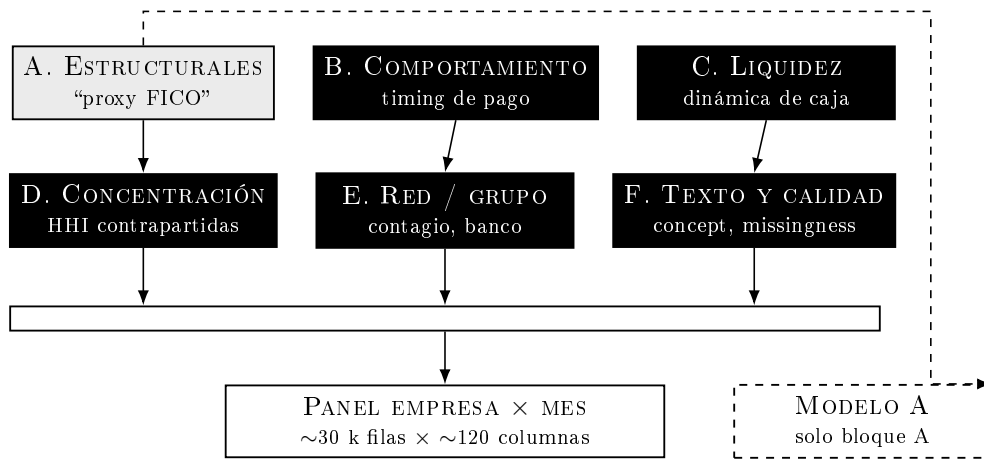


Figure 4: Bloques de variables. El bloque A por sí solo define el *baseline* (Modelo A). Los bloques B–F son la hipótesis del proyecto: información de comportamiento que un scoring crediticio tradicional no observa.

Bloque A — Estructurales (*baseline*, “lo que ve un banco”)

Deuda viva total, ratio *outstanding/granted*, número de productos de deuda, tipo de interés medio ponderado, plazo residual, saldo total, antigüedad de la empresa en plataforma, país, divisa, ERP, tamaño del grupo, número de bancos.

Bloque B — Comportamiento de pago (núcleo de la tesis)

- **DPD** (*days past due*) medio, mediano y p90 por mes: $DPD = \text{payment_date} - \text{due_date}$.
- **Tendencia del DPD**: pendiente de la regresión del DPD mensual sobre los últimos 6 meses. Un nivel alto es un hecho; una pendiente positiva es un *pronóstico*.
- Ratio de facturas **overdue** sobre emitidas, y su derivada.
- **Orden de prelación**: ¿a quién se deja de pagar primero? Retraso en *salary* y *social_security* frente a retraso en proveedores ordinarios. Pagar tarde a un proveedor es negociación; pagar tarde las nóminas es agonía.
- Retraso en **tax**: aplazamiento fiscal como señal temprana.
- Estacionalidad del incumplimiento (¿es estructural o de temporada?).

Bloque C — Dinámica de liquidez

Ratio cobros/pagos mensual y su tendencia; días de caja (saldo/gasto medio diario); volatilidad del saldo; número de días en descubierto; frecuencia de **interest_charge** (síntoma directo de tensión); ciclo de conversión de efectivo aproximado a partir de fechas de factura frente a fechas de cobro.

Bloque D — Concentración

Índice de Herfindahl sobre contrapartidas de cobro:

$$HHI_i = \sum_j s_{ij}^2, \quad s_{ij} = \frac{\text{cobros de } i \text{ desde } j}{\text{cobros totales de } i}$$

Peso del mayor cliente; número efectivo de clientes ($1/HHI$); rotación de la cartera de contrapartidas entre meses (¿está perdiendo clientes?).

Bloque E — Red y grupo

Dado el Hallazgo 2, esta capa no es un GNN, es agregación jerárquica: media de riesgo de las empresas hermanas del grupo *excluyendo la propia* (evita fuga), tamaño del grupo, número de bancos distintos, concentración bancaria, y tasa de deterioro media del banco principal.

Bloque F — Texto y calidad del dato

Los campos **description** y **concept** están anonimizados con *placeholders* pero conservan el vocabulario operativo. Se extraen indicadores de términos de estrés (devolución, impagado, embargo, reclamación) mediante conteo por expresión regular. Además: proporción de transacciones sin contrapartida resuelta, proporción sin conciliación contable, y **tendencia** de ambas — una conciliación que se degrada es un equipo financiero perdiendo control.

Opciones de enfoque

Enfoque	A favor	En contra
Una fila por empresa	Simple	Solo 1.286 muestras: insuficiente
Panel empresa–mes	~30 k muestras; capta trayectorias; permite validación temporal	Requiere cuidado extremo con leakage
Secuencias crudas a red neuronal	Sin <i>feature engineering</i> manual	Con 1.286 entidades, sobreajusta; no explicable
tsfresh automático	Cientos de variables sin esfuerzo	Ruido masivo; ininterpretable ante jurado

Recomendación

Panel empresa–mes con agregaciones en ventanas de 1/3/6/12 meses, y para cada magnitud incluir nivel y tendencia. La tendencia es la que aporta el poder predictivo y la que el balance anual no puede capturar. Evitar generación automática masiva: 120 variables entendibles baten a 4.000 opacas cuando hay que explicar el modelo en cinco minutos.

8 Capa 5 — Entrenamiento y validación

El error que hay que evitar

Con datos temporales, un **train_test_split** aleatorio mezcla el futuro con el pasado y **infla las métricas sistemáticamente**. La validación debe ser estrictamente temporal.



la ventana de validación es siempre posterior a la de entrenamiento

Figure 5: Validación cruzada temporal expansiva. El *holdout* final (últimos meses) se toca una sola vez, al final, y no se usa para seleccionar nada.

Modelos candidatos

Modelo	A favor	En contra
Regresión logística	Interpretable por coeficientes; rápida; <i>baseline</i> obligatorio	Solo relaciones lineales; sensible a escala y outliers
Árbol de decisión	Visualizable, explicable a un no técnico	Inestable; precisión baja
Random Forest	Robusto, pocos hiperparámetros, sin escalado	Peor que boosting en tabular; modelo pesado
XGBoost	Estado del arte en tabular; maneja nulos de forma nativa (clave aquí); <code>scale_pos_weight</code> para desbalanceo	Requiere tuning; puede sobreajustar con pocas muestras
LightGBM	Muy rápido; soporte nativo de categóricas; buen rendimiento con ~30 k filas	Sensible a <code>min_child_samples</code> en clases raras
CatBoost	El mejor con muchas categóricas (bancos, ERP); robusto sin tuning	Entrenamiento más lento
Redes neuronales (MLP/TabNet)	Flexibles	Con 30 k filas tabulares pierden frente a boosting; exigen escalado; opacas
LSTM/Transformer sobre series	Captan dinámica temporal sin <i>feature engineering</i>	1.286 entidades es muy poco; alto riesgo de sobreajuste; difícil de explicar

Recomendación

Regresión logística como *baseline* obligatorio + LightGBM como modelo principal + CatBoost como contraste. Y un *ensemble* por promedio de probabilidades solo si aporta mejora medible: si no, se descarta, porque duplica la complejidad de despliegue a cambio de decimales.

Tratamiento del desbalanceo

Con umbral en el percentil 85, la clase positiva es $\approx 15\%$. Opciones:

- `scale_pos_weight` / `class_weight`: **preferido**, no inventa datos.
- *Undersampling*: descarta información; con 30 k filas no compensa.
- SMOTE: **desaconsejado** en panel temporal — genera empresas sintéticas que violan la estructura temporal y suele empeorar la calibración.

Métricas

- **AUC-PR** como métrica principal (con clases desbalanceadas, el AUC-ROC es engañosamente optimista).
- **Recall@top-k**: de las 50 empresas que el modelo marca como más arriesgadas, ¿cuántas se deterioran de verdad? Es la métrica que un tesorero entiende.
- **Brier score** y curva de calibración: si se muestra “72% de riesgo”, ese número debe significar algo.
- Estabilidad temporal: la métrica por *fold*, no solo la media.

9 Capa 6 — Evaluación y explicabilidad

Función

Demostrar la tesis, no solo reportar un número. Esta capa produce el material de la presentación.

Contenido

- **Tabla comparativa A vs. B** con intervalos de confianza por *bootstrap*: el *lift* es el resultado del proyecto.
- **Ablación por bloques**: entrenar quitando un bloque cada vez. Cuantifica qué aporta cada familia de variables y blinda frente a la pregunta “¿y cómo sabes que es el comportamiento y no el tamaño?”.
- **SHAP**: importancia global (*beeswarm*) y explicación local por empresa (*waterfall*). La explicación local es lo que se enseña en el dashboard.
- **Análisis de errores**: perfil de los falsos negativos. Es la sección que distingue un proyecto serio de una demo.

Opciones de explicabilidad

Técnica	A favor	En contra
Importancia por <i>gain</i>	Gratis, inmediata	Sesgada hacia variables de alta cardinalidad
Permutación	Model-agnostic, honesta	Costosa; engañosa con variables correlacionadas
SHAP (Tree-SHAP)	Exacta y rápida en árboles; local + global; base teórica sólida	Correlación entre variables complica la lectura
LIME	Intuitiva	Inestable entre ejecuciones

Recomendación

TreeSHAP, precalculado en batch y guardado en la base de datos junto al score. No se calcula SHAP en tiempo de petición: se sirve.

10 Capa 7 — Registro de modelos

Función

Versionar el artefacto y sus métricas. Sin esto no se puede responder “¿con qué modelo se generó este score?”.

Contenido mínimo

Hash del *commit*, hash del dataset, semilla aleatoria, hiperparámetros, métricas por *fold*, fecha, esquema de entrada esperado, y el binario serializado.

Opciones

Opción	A favor	En contra
Carpeta versionada + JSON MLflow	Cero dependencias; transparente; suficiente en hackathon UI de experimentos, comparación de runs	Manual; sin UI Servicio extra que mantener en 36 h
W&B	Excelentes gráficas, colaborativo	Requiere cuenta y red

Recomendación

Carpeta versionada con `metadata.json`, y MLflow *solo* si sobra tiempo. Sobre formato: preferir `joblib` para el *pipeline* completo de scikit-learn, y guardar además el modelo en formato nativo del *booster*. Evitar `pickle` desnudo: rompe entre versiones de librería.

Detalle crítico

Lo que se serializa no es el modelo, es el ***pipeline* entero** (imputación, codificación, modelo). Guardar solo el modelo es la causa número uno de *training-serving skew*: el score de producción deja de coincidir con el de validación.

11 Capa 8 — Base de datos servida

Función

Contener lo que el dashboard consulta, ya calculado. El flujo online no recalcula nada.

Tablas

Tabla	Contenido
<code>company</code>	maestro de empresas con atributos de dimensión
<code>company_month_kpi</code>	KPIs mensuales para las series del dashboard
<code>risk_score</code>	score, percentil, banda, versión de modelo, fecha
<code>score_explanation</code>	top- <i>n</i> contribuciones SHAP por empresa
<code>benchmark</code>	agregados p25/p50/p75 por sector, tamaño y país

Sobre el dashboard de benchmarks

Un benchmark solo es honesto si el grupo de comparación es homogéneo. Comparar una empresa de 5 M€ con una de 50 M€ no dice nada. Se recomienda definir cohortes por **tamaño (cuartil de volumen de transacciones) × país × tamaño de grupo**, y **no mostrar un benchmark cuando la cohorte tenga menos de 10 empresas** — con 1.286 empresas y tres dimensiones de corte, muchas celdas quedan vacías.

Recomendación

Vistas materializadas para los agregados de benchmark, refrescadas por el job batch. Índices sobre (`company_id`, `date`). Nada de calcular percentiles en la petición HTTP.

12 Capa 9 — Servicio de inferencia

Función

Exponer el modelo. Dos modos, y conviene implementar ambos porque responden a preguntas distintas.

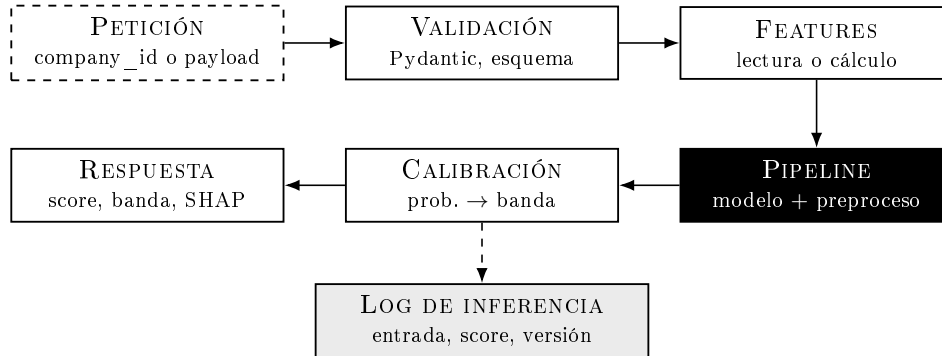


Figure 6: Flujo de una petición de scoring. La calibración es un paso explícito: el modelo devuelve una puntuación, el producto devuelve una probabilidad interpretable.

- **Batch:** recalcula todas las empresas y vuelca a `risk_score`. Es el camino normal y el que alimenta el dashboard.
- **Online:** `POST /score` con datos de una empresa. Es el que impresiona en la demo (“mueve este parámetro y mira cómo cambia el riesgo”) y el que permite simulación de escenarios.

Opciones

Opción	A favor	En contra
FastAPI	Async, validación con Pydantic, OpenAPI automático	—
Flask	Mínimo, muy conocido	Sin validación ni docs automáticas
BentoML / Seldon	Pensados para servir ML	Sobredimensionado para un hackathon
Modelo embebido en el frontend	Sin servicio extra	Imposible versionar; mezcla capas

Recomendación

FastAPI con el *pipeline* cargado una sola vez al arrancar (no por petición). Endpoint `/health` que devuelva la versión del modelo cargado: es lo primero que se mira cuando algo falla en la demo en directo.

13 Capa 10 — API / Backend

Función

Separar la lógica de negocio del servicio de modelo. Mezclarlos es el error que impide sustituir el modelo sin tocar el producto.

Endpoints

Endpoint	Devuelve
GET /companies	listado paginado con filtros
GET /companies/{id}	ficha: KPIs, productos, series
GET /companies/{id}/score	score actual, histórico, explicación
GET /benchmarks	percentiles por cohorte
POST /simulate	score bajo escenario hipotético
GET /model/info	versión, métricas, fecha de entrenamiento

Recomendación

Un único servicio FastAPI que integre capas 9 y 10 en el hackathon (menos contenedores, menos cosas que fallen en directo), pero con **módulos separados** en el código, de modo que la división sea trivial después. /model/info público desde el principio: da credibilidad y facilita depurar.

14 Capa 11 — Frontend

Función

Traducir probabilidades en decisiones. El criterio de diseño es simple: un tesorero debe poder responder “¿de qué me preocupo hoy?” en menos de diez segundos.

Vistas

1. **Cartera**: tabla ordenada por riesgo, con delta frente al mes anterior. El delta importa más que el nivel: una empresa que sube veinte puntos es más urgente que una que lleva seis meses alta.
2. **Ficha de empresa**: evolución del score, serie de liquidez, DPD, y explicación SHAP en lenguaje natural (“el retraso medio de pago ha pasado de 4 a 23 días”).
3. **Benchmarks**: posición frente a la cohorte, con el tamaño de cohorte visible.
4. **Simulador**: escenarios (“si el cliente principal paga 30 días más tarde”).
5. **Rendimiento del modelo**: A vs. B, curvas, ablación. Es la vista que convence al jurado técnico.

Opciones

Opción	A favor	En contra
Streamlit	Muy rápido de escribir; ideal si el equipo es todo ML	Estética limitada; recarga completa; no parece producto
React + Vite	Aspecto de producto real; control total	Requiere alguien de front en el equipo
Dash	Integrado con Plotly	Menos flexible que React
Next.js	SSR, routing	Complejidad innecesaria aquí

Recomendación

Depende del equipo, y conviene decidirlo hoy, no el sábado: **si hay alguien cómodo con React, React + Vite + Recharts** — el reto lo plantea una empresa de producto y la diferencia visual se nota. **Si no, Streamlit sin complejos** y se invierte el tiempo ahorrado en las capas 3 y 4, que es donde está el fondo del proyecto.

15 Capa 12 — Infraestructura, reproducibilidad y MLOps

Mínimos exigibles

- `docker-compose` con tres servicios: `postgres`, `api`, `frontend`.
- **Un solo comando** que reconstruya todo desde los CSV: `make all`. Si el jurado no puede reproducirlo, no existe.
- Semillas fijadas en todas partes.
- Variables de entorno en `.env`, nunca credenciales en el repositorio.
- CI en GitHub Actions: `lint` + tests + un *smoke test* que entrene sobre una muestra pequeña y verifique que el *pipeline* completo no se rompe.

Recomendación sobre el cloud

Desplegar en OCI o similar **solo si el reto lo puntúa**. En caso contrario, todo en local con Docker: un despliegue a medias que falla en la demo cuesta más de lo que suma. Alternativa razonable: local para la demo + una URL pública de respaldo.

16 Riesgos y decisiones abiertas

Riesgo	Impacto	Mitigación
Fuga de etiqueta	Métricas irreales; se cae en preguntas	Separación estricta de ventanas; revisión cruzada de la lista de <i>features</i>
Target arbitrario	Todo el modelo queda en el aire	Documentar <i>D</i> ; análisis de sensibilidad al umbral (85 vs. 90)
Datos sintéticos	Las correlaciones pueden ser artefactos del generador	Declararlo abiertamente; validar que las relaciones tienen sentido económico
Muestra pequeña (1.286)	Varianza alta entre <i>folds</i>	Intervalos por <i>bootstrap</i> ; no presumir de decimales
Sobreingeniería	No llegar a tener demo	Congelar alcance; <i>baseline</i> funcionando antes de optimizar
Grafo inexistente	Perder tiempo en un GNN inútil	Ya descartado con evidencia (Hallazgo 2)

Decisiones que faltan por cerrar

1. Umbral del target: percentil 85 o 90.
2. Longitud de la ventana de resultado: 3 o 6 meses.
3. Streamlit o React.
4. Si el simulador entra en el alcance o queda como ampliación.
5. Si se despliega en cloud.

17 Orden de construcción sugerido

El orden no es el de los capítulos. Se construye primero el camino completo más fino posible, y luego se engorda.

1. Ingestión a Parquet + informe de calidad.
2. Panel empresa–mes con **diez** variables, no ciento veinte.
3. Etiqueta v1 y regresión logística. **Aquí ya hay número que enseñar.**
4. Bloques B–F completos + LightGBM. Aquí aparece el *lift*.

5. SHAP + ablación. Aquí aparece la narrativa.
6. API y persistencia de scores.
7. Frontend.
8. Benchmarks, simulador, despliegue — por este orden de prioridad.

La regla es tener algo demostrable desde el paso 3. Un sistema completo y modesto gana a un modelo brillante sin interfaz.